Practical LLM Use Cases for Security

...

by Jonathan Marcil

ConFoo

February 2026

Overview of boring use cases for security

This talks tackle the unappealing for individuals but important for the organizations, with a focus on what matters for developers.

- Security Policy
- Security Training
- Security Code Review

Why boring?

Security is intangible by itself, and highly context dependant.

The results are not direct, immediate.

The goal of reducing risk for the organization doesn't apply to the individual.

Sometimes it's about the latest and greatest things, but more often it's about repeating the same old things.

What’s not boring right now

Web Application Security Up-to-date: The 2025 OWASP Top Ten

Christian Wenz

Talk Nerdy to Me: Building an MCP Server with Node.js

Scott Stroz

How I automated my life with MCP Servers

Cedric Clyburn

Why stay then?

Well, this has about one year of work experience, with real AI and real humans, shared with you.

It’s also presenting novel approaches, that could be ground breaking for your organization.

LLM Uses Cases

Security Policy

Defines the parameters for execution within an organization by mapping business needs into requirements.

LLM for Security Policy

Context

Mid-size business doing high velocity projects for external clients.

Security Policies were needed for compliance with clients and written.

Some cloud enforcements were made, but no SDLC.

Goal

Nobody had time to read the policies, an in-context way to inform people was needed due to the high velocity (no time to stop and read policies).

Answer clients questionnaires about security policy.

Implementation

An LLM chat bot that would have all the policies loaded and would be able to answer questions for the developers.

It answers direct questions about policy but also can guide development.

What was tried as a prototype

Chat bot in Streamlit	• Would show history but not use it in context • Copy paste for questionnaires
RAG	• Naive Text Indexer; llama_index Splitter • ChatGPT 4 $t=0$ with generic prompts
CAG	• Gemini 2.5 Pro $t=0.5$ for answers • ChatGPT 4o $t=0.5$ judge to select best of 4
RAG v2	• Added Security Policies detailed prompt • Ran tests in MLflow to confirm results

What's RAG?

Retrieval Augmented Generation

When loading information, you index your content as vectors and store it in a vector database.

When answering questions, the chat bot retrieves it as human readable text to add into the LLM context.

What's CAG?

Context Augmented Generation

You just load your content into the context (prompt) of the LLM.

At the time (last year) was rather new for larger content because models started to handle bigger contexts that fits more content.

What's a judge?

An LLM that is asked to judge something based on criterias.

Context include a system prompt that tells it it's a judge.

In our case, the context also included the same CAG (with our policies) as the judged LLM. The judge would approve a response and then regular code selected the highest confidence level response returned by the judged LLM.

Winning

LLM answers better than human

For the pilot that we did, and the past questionnaires answers, we saw that I was agreeing more with the results from the Chat bot than the humans.

To their defence, the humans didn't read the policies because they didn't exists at the time for some of the answers.

Chat bot provided logs

Regular humans would read the policies generated page view logs (that's how I know nobody was reading them unless I sent them the link during a discussion), but with the LLM chat bot we got logs of questions that were looked up and answers that were most likely utilized.

This could then be used bench the bot answers but also find gaps in policies.

Failing

Lack of accountability

Turns out that beyond people not trusting an LLM giving the right answers (even if our benchmark would show it was better than humans), accountability was important to management and it was felt like being offloaded to the LLM.

Context switch still needed

While filling security questionnaires could have been easily be filled with the proper tooling, copy pasting each question was tedious.

But for the devs guidelines part, they would still need to open the tool and actively ask a question.. almost no one does that by themselves.

Failing

Tests needed more time

To get a very good level of confidence from a professional standpoint, we needed thousands (1000+) of questions/answers, while we were working out of 100 due to decision to not commit more time to prototype.

So while our results were great, they weren't acceptable scientifically.

Not in use

We decided to not further invest for now because of other priorities for AppSec.

I’m kind of the sole users sometimes when I look things up because I like it better than Jira search or their AI.

Security Training

Educate the devs on how to design secure systems and write secure code.

Security Training: Content Creation

Context

Application Security expertise support for various cybersecurity startups.

This was on-going for more than half a year to get know the startups.

Goal

Create a training for developers of the various startups.

Create supporting documents.

Implementation

Knowledge was brainstormed and brain-dumped as it in various .md files totally more than 60kb / 14,000 tokens.

And loaded as CAG in various LLM based tools to produce deliverables.

Content Creation Deliverables

Guide for Startups	• Originally goal was to do the editing myself • Gemini 2.5 Pro to generate static HTML
Multi-LLM Chat Bot	• Internal testing to bench the model manually • o4-mini, gpt-4.1 t=0.5,1.0, gpt-4o, 4.1 no CAG
Demo micro-sites	• PDF guide, but interactive mini-sites • Gemini 2.5 Pro (Web & AI Studio)
Training content	• Helper to reorganize knowledge into modules • HTML Slide decks for the training modules

Warning

LLM was used to reorganize content from braindump, not as a knowledge base itself.

This is an important nuance.

—

Multi-LLM Chat Bot

How can I test for application security?

gpt-4o | o4-mini | gpt-4.1 (t=1) (without knowledge) | gpt-4.1 (t=0.5) | gpt-4.1 (t=1)

Pour tester la sécurité applicative (« application security ») dans le contexte d'une startup, tu as plusieurs méthodes disponibles, selon ta phase et ton niveau de maturité. Voici une réponse adaptée aux équipes de dev, CTO, ou tech leads :

1. Tests automatisés dans le CI/CD

- **Scanners de vulnérabilités dans le code (SAST):** Utilise des outils comme *CodeQL*, *Find Security Bugs* (Java), *ruff & Bandit* (Python), ou *Security Code Scan*

Type your message here...

Send

Micro-site demo

https://wonderful-mushroom-0d4dc120f.1.azurestaticapps.net/demo9-5d.html

Micro-site demo

cyber eco **Sécurité Applicative pour les Startups**

Phase "Validation":

Preuve de Concept et MVP

Priorité SecApp : Minimale mais essentielle

Il faut protéger le MVP et les premières données de test. L'enjeu principal est d'éviter une exposition non désirée des idées et du code.

Actions recommandées

- **Gestion des Identités et Accès (GIA) :** Pour les démos, utilisez un portail captif d'un fournisseur cloud pour ajouter manuellement les utilisateurs autorisés.
- **Architecture et Design Sécurisés :** Intégrez les principes fondamentaux de SecApp pour guider la conception. Un design simple est souvent un design plus sécuritaire.
- **Protection de l'infrastructure :** Utilisez les composants de sécurité offerts par votre cloud (WAF, Gateway) pour protéger le MVP sans avoir à coder ces fonctionnalités.

← Votre priorité

Posez une question... Envoyer

Winning

Saved a ton of editing time

Writing process is to braindump first and then edit. It took about 1 week to braindump, and would have taken 2 weeks to generate the guide. With LLM using the braindump as input, it took 1 or 2 days.

Generating different output is easy

You can “farm” the output style that you want by generating multiple versions and the LLM is going to reorganize the content for it quickly.

This allow for fast prototyping.

It can also do the same to the semantic and overall structure of the knowledge.

Winning

CAG worked very well

The content generated was extremely close to what I would write myself as the knowledge base original writer.

Being the subject matter expert myself I could bench very easily the results.

Generating slides as HTML

This allowed for a versioning in Git that works better than Google Slides.

It choose Reveal.js which was already my favorite framework to make a slideshow.

Failing

The trainings module needed manual work

Something wasn’t written in the knowledge base about the priority and the selection of the content, so there’s no way the LLM could have gotten this right.

I ended up doing things manually.

HTML slide content was bad

The ordering and structure of the generated slides for the training was bad, unnatural.

The content of the slides were lacking as well.. But that’s because the knowledge pointed to references that weren't included.

Also the knowledge was too broad for something as specialized as the slides.

So in the end, all slides were done manually.

Security Training: LLM Assistant

Answer devs questions about detailed content from the training itself.

Include references to knowledge base

Instead of just having the brainstorm loaded into the LLM context, the slidedeck, code examples and references documentation were loaded.

This unleashed the LLM power to a point that it was now good at more precise answers.

Fully loaded LLM features

Dynamic augmentation	• A judge determines what the question is about • Documentation is loaded based on judgement
Specialized Chat Bot	• Web interface to Q&A training content • Using judge and RAG gpt-4.1-mini, CAG gpt-4.1
Helper micro-sites	• Specialized content for some training • Gemini 2.5 Pro manually loaded docs in CAG
Visuals Cards	• Generate images related to training content • Nano Banana docs in CAG (Google AI Studio)

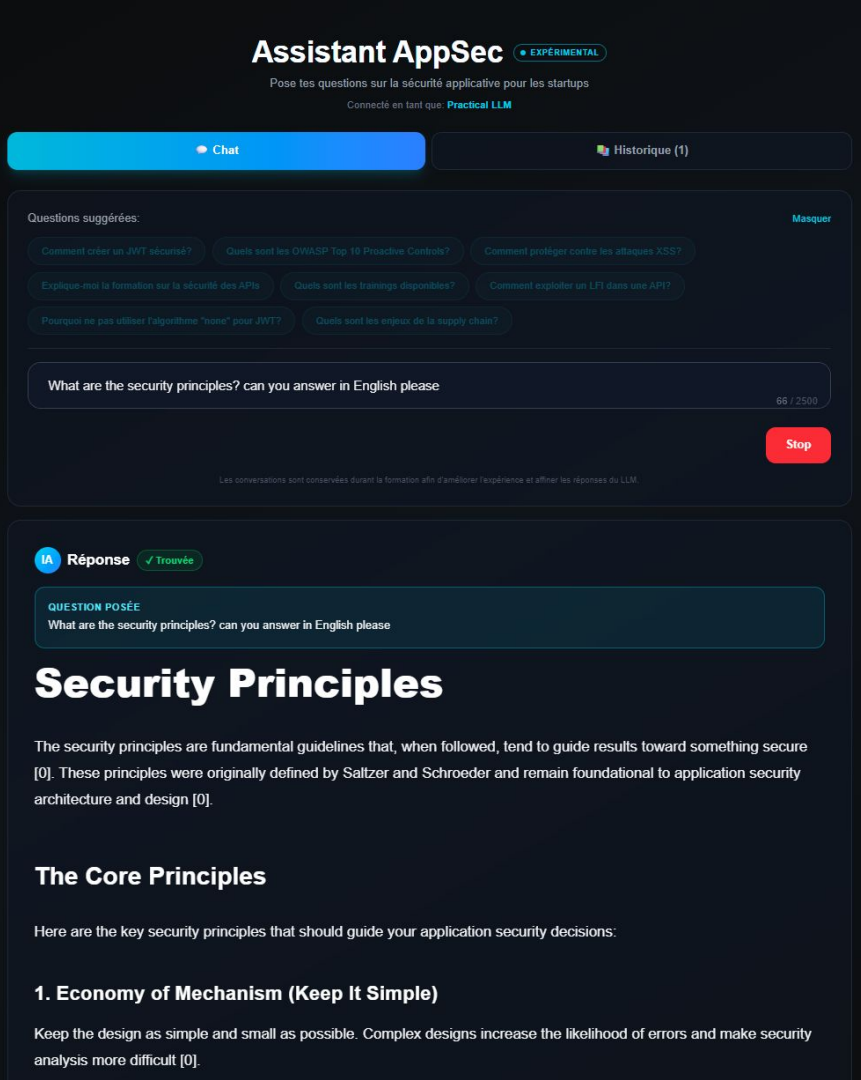
AppSec Assistant

Answering question about the training agenda

AppSec Assistant

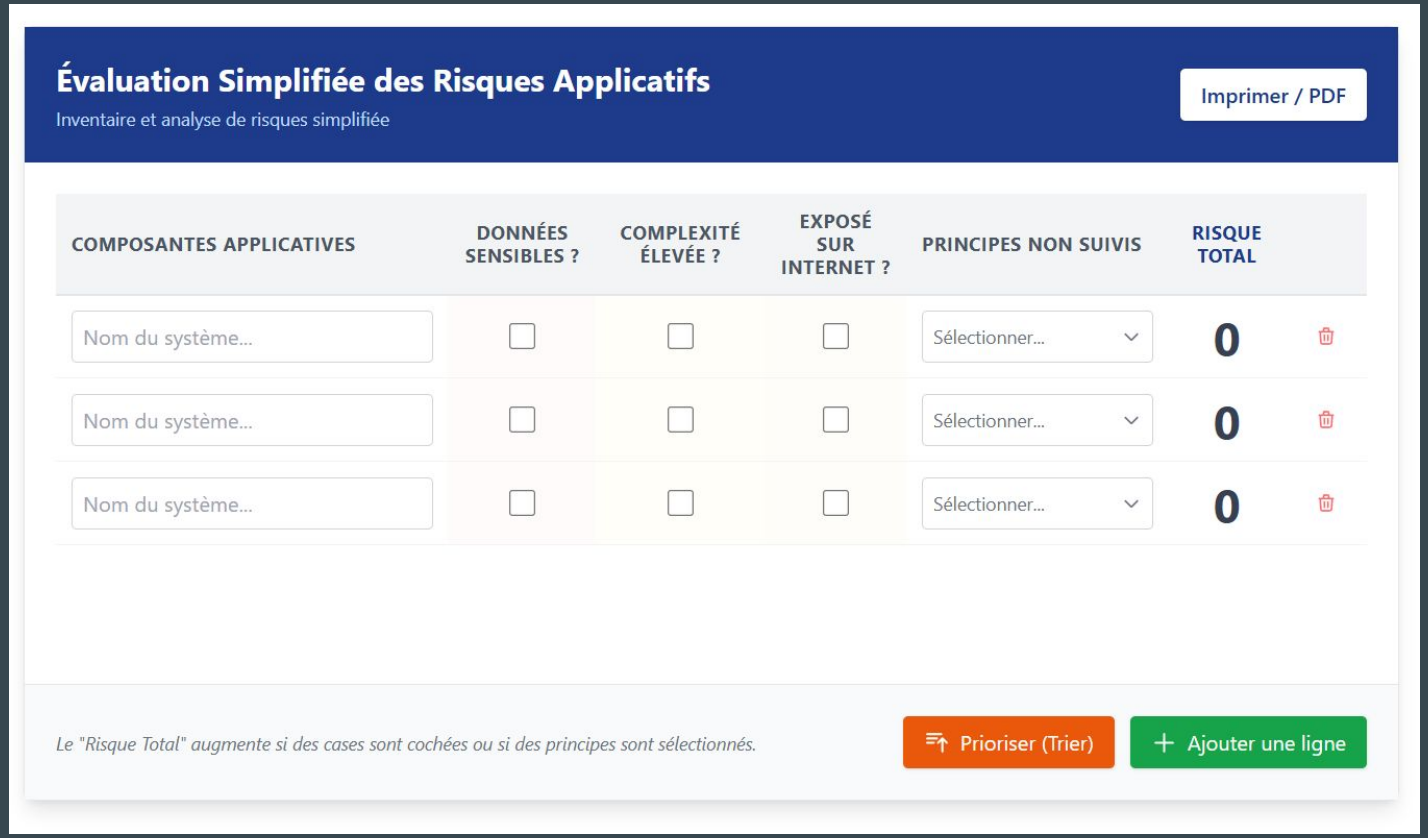
Answering question about the specific content referenced during training (security principles).

The assistant can answer in any languages of course!



Helper micro-site: Simplified Risk Rating

Prompt was about creating the micro-site, and then “Principes non suivis” were loaded with content the LLM knew from context.



Helper HTML page

Reorganize content quickly into another format better for consumption for the trainees than the slidedeck itself.

Instant multi-modal generation

Liste des Outils de Sécurité Applicative

Voici une liste de référence des outils open source présentés lors de l'atelier, regroupés par catégorie.

Les slides de l'atelier sont disponibles ici.

SCA: Software Composition Analysis

Trivy

- **Description :** Détecte les vulnérabilités (CVEs) dans les images Docker, les systèmes de fichiers et les dépôts Git.
- **Utilisation :** Idéal pour scanner les dépendances de vos applications (ex: `package-lock.json`, `pom.xml`, etc.).
- **Avantage :** Peut générer un SBOM (Software Bill of Materials) pour la conformité.

Dependabot

- **Description :** Un service qui scanne vos dépendances et crée automatiquement des Pull Requests pour les mettre à jour vers des versions sécuritaires.
- **Utilisation :** Intégration native à GitHub. Configurez-le via un fichier `dependabot.yml` dans votre dépôt.
- **Avantage :** Actionnable immédiatement. Il ne fait pas que signaler un problème, il propose une solution (la PR).

OpenSSF Scorecard & Open Source Insights

- Ces outils vont au-delà du scan de CVEs. Ils évaluent les pratiques de sécurité d'un projet open source (ex: a-t-il des revues de code ? Du 2FA pour les mainteneurs ?).
- Utiles pour choisir une nouvelle dépendance ou pour auditer les dépendances critiques de votre projet.

Visuals Cards

Based on items in the main knowledge and augmentation.

PRINCIPE DE SÉCURITÉ 1

Économie De Mécanisme

KISS: Keep It Simple and Stupid

La simplicité réduit les risques. Un design simple est plus facile à inspecter et à sécuriser, car les erreurs sont moins probables.

PRINCIPE DE SÉCURITÉ 2

Sûreté Par Défaut

Deny all by default, then allow some

Refuser l'accès par défaut et n'autoriser que ce qui est explicitement permis. Une erreur mène à un refus plutôt qu'à un accès.

PRINCIPE DE SÉCURITÉ 3

Médiation Complète

TOCTOU attack protection

Chaque demande d'accès à une ressource doit être validée, à chaque fois, sans exception. On ne se fie jamais à une décision précédente.

PRINCIPE DE SÉCURITÉ 4

Conception Ouverte

No security by obscurity

La sécurité ne doit pas reposer sur le secret du design, mais sur la robustesse des clés cryptographiques et de l'implémentation.

PHASE D'ÉVOLUTION 1

Idéation

PHASE D'ÉVOLUTION 2

Validation

PHASE D'ÉVOLUTION 3

Lancement

Winning

LLM Chat Bot was great

Only sparse manual testing, but all of the answers were spot on, as the training intended to say.

Regular Chat Bot with same model vs. Dynamic Augmentation gave very different results.

Failing

Not used as much during training

Participant were focus on the actual talk and the exercice, and the LLM assistant wasn't very much used due to that.

However, usage after the training to revise things we saw, or even answer question for people that are interested about what they would learn during training is promising.

Security Code Review

Utilize LLM to review code based on training content.

Why not use an LLM for code review?

At some point during the training preparation, the idea came out to use the documentation augmented version of the LLM to generate prompts to use for code review.

Then during training, the hands on part of a module was about that.

Generated Prompt for reviewing Security Principle 1

System Prompt: Security Principle 1 - Economy of Mechanism
Role:
Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat Economy of Mechanism, lorem ipsum dolor sit amet, consectetur adipiscing elit.

Objective:
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua unnecessary complexity lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor.

1. Core Principle Definition
Economy of Mechanism: "Lorem ipsum dolor sit amet, consectetur adipiscing elit."

The Rationale: Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi.
The Risk: Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
The Startup Context: Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco.

2. Analysis Framework
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua:

A. Code-Level Complexity (The "Spaghetti" Filter)
Abstraction Overkill: Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
Security Locality: Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor.
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.

Language Specifics:
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.

B. Architectural Complexity (The "Over-Engineering" Filter)
Service Boundaries: Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

Infrastructure: Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua?
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt.

C. Dependency & Supply Chain (The "Bloat" Filter)
Third-Party Risk: Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor.
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod.
Lorem ipsum dolor sit amet, consectetur adipiscing elit.

3. Instructions for Response
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore:

1. The Complexity Audit
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt.
Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.

2. The "Economy" Refactoring
Code: Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor.
Architecture: Lorem ipsum dolor sit amet, consectetur adipiscing elit.
Tools: Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor.

3. Verification Strategy
Lorem ipsum dolor sit amet, consectetur adipiscing elit.
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt.

4. Tone and Style
Lorem ipsum dolor sit amet, consectetur adipiscing elit.
Lorem ipsum dolor sit amet, consectetur adipiscing elit.
Lorem ipsum dolor sit amet readability lorem ipsum dolor sit amet.
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor.

5. Constraint Checklist
Lorem ipsum dolor sit amet, consectetur adipiscing elit:
Lorem ipsum dolor sit amet, consectetur adipiscing elit?
Lorem ipsum dolor sit amet, consectetur adipiscing elit?
Lorem ipsum dolor sit amet, consectetur adipiscing elit?
Awaiting Input: Please provide code, architecture descriptions, or configuration files for analysis.

Micro-site: Prompts download and code sample

Security Code Review with an LLM Slides deck

PROMPTS LIBRARY

- OWASP C1: Implement Access Control
- OWASP C2: Cryptographic Protection
- OWASP C3: Input Validation
- OWASP C4: Security by Design
- OWASP C5: Secure Configuration
- OWASP C6: Supply Chain Security
- OWASP C7: Secure Digital Identities
- OWASP C8: Browser Security
- OWASP C9: Logging & Monitoring
- OWASP C10: Stop SSRF
- Principle 1: Economy of Mechanism
- Principle 2: Fail-Safe Defaults
- Principle 3: Complete Mediation

Principle

Principle 1: Economy of Mechanism

DESCRIPTION

Analyze code/architecture to identify unnecessary complexity that hinders security verification. Guide users toward 'Simplicity by Design' so protection mechanisms are small and verifiable.

SUMMARY

Complexity is the hiding place of vulnerabilities. This prompt instructs the LLM to act as an architect auditing for over-engineering, abstraction overkill, and excessive dependencies. It focuses on refactoring code to be flatter and simpler (KISS) to make security verification possible.

Download .md File

VULNERABLE CODE SAMPLES

- Exploit 1: Python Pickle Deserialization
- Exploit 2: Local File Include (LFI)
- Exploit 3: Signature Forgery
- Exploit 4: Timing Attack
- Exploit 5: Auth Bypass
- Exploit 6: Frontend Secrets
- Damn Vulnerable NodeJS Application
- Broken Crystals
- OWASP Juice Shop: API Routes
- OWASP Juice Shop: Database Models
- OWASP Juice Shop: Angular Frontend
- CloudGoat: Data Factory Secret Theft
- CloudGoat: Beanstalk Secrets

Winning

AWESOME FAST RESULTS

In less than 15 minutes, teams found vulnerabilities. Some fixed findings during the session.

Turns out it's very easy to just dump a big prompt into their day to day LLM tooling.

People knew about the content because it was fresh in their mind from the past day of training, so the LLM answer were understood better.

Failing (sort of)

People picked the first prompt

We perhaps gave too many prompts (18) to people so they were picked was was on top.

What was on top was high level principles, made for architecture review, but results were still good.

We could assume that people understood maybe an non-optimal framing with prior understanding of concepts is better than optimal framing without prior understanding.

LLM Code Review: Findings samples

Test endpoint in production	Impact confirmed, fixed during training.
Excessive API permissions	Impact confirmed.
Secret found in code	No real impact because unused secrets.
API handler issue	Undisclosed details, confirmed as critical impact.

Thanks!

Slides on:

about.jonathanmarcil.ca

Special thanks to:

Cybereco

IVADO Labs

Confoo 2026

Camile

feedback form